

AGILE DUCKS SERVICE DESK BLUEPRINT

SOFTWARE ARCHITECTURE AND MINIMALIST DARK AESTHETIC GUIDE

1. VISUAL IDENTITY & AESTHETIC GUIDE (THE DUCK CONCEPT)

The project adopts a minimalist, modern, high-contrast user interface layout running entirely on an extreme dark color scheme, heavily inspired by the natural feathers of a **Mallard Duck** combined with the classic software engineering philosophy of "**Rubber Ducking**".

The Rubber Duck Principle (UX): When writing a ticket, the React front-end will subtly guide the user under the concept of *Rubber Duck Debugging*, prompting them to explain their problem step-by-step. This directly enhances technical report quality and documentation integrity from its inception.

Interface Color Palette

- **Main Background (Extreme Dark Mode):** #0B0F12 (Obsidian Slate). Engineered to prevent eye strain and maintain a premium layout look.
- **Primary Accent Color:** #0D5C4D (Mallard Teal). Reserved for core call-to-action UI buttons, active states, and focus indicators.
- **Critical Alert Color:** #F2A900 (Beak Gold). Exclusively used for critical internal priorities, high-urgency notifications, and near-breach SLA timers.

2. SYSTEM VISION & ARCHITECTURE STRATEGY

To avoid over-engineering during initial development stages, the backend follows a strictly decoupled **Layered Monolithic Architecture**, managing data isolation constraints within Django REST Framework (DRF) handlers to secure clear paths towards multi-node microservices later on.

Day-1 Scalability Safeguards:

- **End-to-End UUIDv4 implementation:** Every single database entity utilizes explicit non-sequential primary keys to completely stop cross-resource enumeration risks and support zero-friction migrations.
- **Decoupled Database Scope Filtering:** Multi-tenant or area data barriers are enforced directly by altering the ORM queryset runtime generator, leaving no gaps for unexpected information exposure.

3. CORE DATABASE SCHEMA & DOMAIN MODELING (DJANGO ORM)

The following blocks outline the core entity relationships managing structural user scopes across distinct organizational areas alongside deep ticketing state flows.

A. Users and Profile Architecture (**users** app)

```
import uuid
from django.db import models
from django.contrib.auth.models import User

class Area(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    name = models.CharField(max_length=100, unique=True)
    created_at = models.DateTimeField(auto_now_add=True)

class UserProfile(models.Model):
    ROLE_CHOICES = [
        ('CLIENT', 'Client / Submitter'),
        ('RESOLVER', 'Staff / Resolver'),
        ('SYSADMIN', 'System Administrator'),
    ]
    user = models.OneToOneField(User, on_delete=models.CASCADE, related_name='profile')
    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='CLIENT')
    areas = models.ManyToManyField(Area, related_name='users')
```

B. Core Ticketing Engine (**tickets** app)

```
class Ticket(models.Model):
    URGENCY_CHOICES = [('LOW', 'Low'), ('MEDIUM', 'Medium'), ('HIGH', 'High')]
    PRIORITY_CHOICES = [('LOW', 'Low'), ('MEDIUM', 'Medium'), ('HIGH', 'High'), ('CRITICAL',
'Critical')]
    STATUS_CHOICES = [('OPEN', 'Open'), ('IN_PROGRESS', 'In Progress'), ('RESOLVED',
'Resolved'), ('CLOSED', 'Closed')]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    title = models.CharField(max_length=200)
    description = models.TextField()

    created_by = models.ForeignKey(User, on_delete=models.PROTECT,
related_name='created_tickets')
    source_area = models.ForeignKey(Area, on_delete=models.PROTECT, related_name='area_tickets')
    assigned_to = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True,
related_name='assigned_tickets')

    urgency = models.CharField(max_length=10, choices=URGENCY_CHOICES, default='MEDIUM')
    internal_priority = models.CharField(max_length=10, choices=PRIORITY_CHOICES,
default='MEDIUM')
    status = models.CharField(max_length=15, choices=STATUS_CHOICES, default='OPEN')

    estimated_resolution_time = models.DateTimeField(null=True, blank=True)
    resolution_documentation = models.TextField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

4. ACCESS CONTROL & AUTHORIZATION MATRIX

Data exposure barriers ensure the endpoint controller outputs exclusively the dataset matching the caller's organizational privileges.

Role / Functional Map	Create Ticket	View Context Scope	Update Internal Priority	Assign Staff
System Administrator	Optional	Global Scope (All assets/tickets)	Yes	Yes
Staff / Resolver	Optional	Assigned Only (Personal queue)	No	No
Client / Submitter	Yes	Area Restricted (Bound to areas)	No	No

Dynamic Scope Enforcement Pipeline (DRF ViewSet Handler)

```
# tickets/viewsets.py
from rest_framework import viewsets, permissions
from .models import Ticket

class TicketViewSet(viewsets.ModelViewSet):
    serializer_class = TicketSerializer
    permission_classes = [permissions.IsAuthenticated]

    def get_queryset(self):
        user = self.request.user
        profile = user.profile

        if profile.role == 'SYSADMIN':
            return Ticket.objects.all().select_related('created_by', 'source_area',
            'assigned_to')

        if profile.role == 'RESOLVER':
            return Ticket.objects.filter(assigned_to=user).select_related('created_by',
            'source_area')

        if profile.role == 'CLIENT':
            user_authorized_areas = profile.areas.all()
            return
            Ticket.objects.filter(source_area__in=user_authorized_areas).select_related('created_by',
            'source_area')

        return Ticket.objects.none()
```

5. ROADMAP EVOLUTION TO FULL WORK MANAGEMENT

This early structural blueprint lays out modular upgrade strategies to safely pivot into comprehensive agile task coordination environments:

- **Service Desk Upgrade:** The static time tracking model will be superseded by automated, Celery-backed SLA monitoring pipelines checking target thresholds continuously based on urgency variations.
- **Work Management Upgrade:** "Areas" will widen into cross-functional project entities, converting standard statuses into customizable state-machine graphs governing interactive task board components.